



Lab Assignment –8 (Applied Normalization & Advanced Concurrency)

Department: CSE

Subject: DIS Lab
Code: 22CPP217

Objective: Transform a poorly designed, denormalized database into a BCNF-compliant schema using SQL data migration. Then, master multi-user session management by intentionally causing Database Deadlocks and observing Phantom Reads.

Pre-Lab Setup: Run this block to initialize the environment.

SQL

-- 0. Reset Environment

SET SQL_SAFE_UPDATES = 0;

DROP TABLE IF EXISTS Bad_Orders, Accounts;

-- 1. Denormalized Schema for Normalization Practice

```
CREATE TABLE Bad_Orders (  
    Order_ID INT,  
    Product_ID INT,  
    Customer_ID INT,  
    Customer_Name VARCHAR(50),  
    Customer_City VARCHAR(50),  
    Product_Name VARCHAR(50),  
    Category VARCHAR(50),  
    Quantity INT,  
    Unit_Price DECIMAL(10, 2),  
    Order_Date DATE,  
    PRIMARY KEY (Order_ID, Product_ID)  
);
```

INSERT INTO Bad_Orders VALUES

(1, 1, 101, 'Alice', 'Jaipur', 'Laptop', 'Electronics', 1, 50000.00,
'2024-03-01'),

(1, 2, 101, 'Alice', 'Jaipur', 'Smartphone', 'Electronics', 1, 20000.00,
'2024-03-01'),

```
(2, 1, 101, 'Alice', 'Jaipur', 'Laptop', 'Electronics', 1, 50000.00,
'2024-03-05'),
(3, 3, 102, 'Bob', 'Delhi', 'Tablet', 'Electronics', 1, 30000.00,
'2024-03-10');
```

-- 2. Banking Setup for Concurrency

```
CREATE TABLE Accounts (
    Account_ID INT PRIMARY KEY,
    Account_Name VARCHAR(50),
    Balance DECIMAL(10, 2)
);
```

```
INSERT INTO Accounts VALUES
(201, 'Vikram', 10000.00),
(202, 'Sanya', 15000.00);
```

Q1: The Bad_Orders table is in 1NF but suffers from severe redundancies. Execute the following tasks to programmatically demonstrate these flaws.

- **Update Anomaly:** Write an UPDATE query changing Alice's city to 'Mumbai', but restrict it only to Order_ID = 1. Next, write a SELECT query using GROUP BY and HAVING COUNT(DISTINCT Customer_City) > 1 to programmatically expose any customer with conflicting city data.
- **Deletion Anomaly:** Write a DELETE query to remove Order_ID = 3 (Bob's order). Then, write a SELECT query attempting to fetch the Category and Unit_Price for the product 'Tablet'.
 - Query returns 0 rows, proving the product data was destroyed along with order.
- **Insertion Anomaly:** Write an INSERT statement attempting to add a new product (Product_ID = 4, 'Monitor', 'Electronics', 15000) without assigning it to an order (leave Order_ID and customer fields NULL).
 - Execute it and record the exact MySQL Error Code and message that blocks the transaction.

Q2: Decompose the table into 3NF.

(Important: Before starting Q2, re-run the Pre-Lab Setup code block at the top of this page to undo the anomalies you caused in Q1 and reset Bad_Orders to a clean state)

1. Write CREATE TABLE statements for four new normalized tables: Customers, Products, Orders, and Order_Items. Ensure you assign appropriate Primary Keys and Foreign Keys, and create parent tables before child tables.
2. Write INSERT INTO ... SELECT DISTINCT statements to automatically extract the data from Bad_Orders and populate your four new tables without duplicating data.
3. **Lossless Verification** : Write a single SELECT query utilizing JOIN across all four new tables. The resulting output must exactly match the original Bad_Orders table in both columns and rows, programmatically proving a lossless decomposition.

(Q3 & Q4 Pre-Check (Run in both tabs): SELECT CONNECTION_ID(); *If the numbers match, you are sharing a session and Part B will fail. You must open a completely new, independent connection window.)*

Q3: When two concurrent transactions attempt to acquire exclusive locks on the same resources but in reverse order, a deadlock occurs and the engine must intervene.

1. **Session A:** START TRANSACTION; Update Vikram's balance (Account 201) to 9000. Do not commit.
2. **Session B:** START TRANSACTION; Update Sanya's balance (Account 202) to 14000. Do not commit.
3. In **Session A**, attempt to reduce Sanya's balance by another 1000.
4. In **Session B**, While Session A is hanging, attempt to update Account 201.

CLEANUP: Type ROLLBACK; in BOTH sessions first to clear all locks before moving to Question 4.

Q4: A Phantom Read occurs when new rows are added to a table while you are in the middle of reading it.

Part A: Triggering the Anomaly

1. **Session A:** Set isolation level to READ COMMITTED. Start a transaction and run SELECT SUM(Balance) FROM Accounts;. Record the sum.
2. **Session B:** Start a transaction. Insert a new account (203, 'Kabir', 5000.00) and COMMIT.
3. **Session A:** Run the exact same SELECT SUM(Balance) query again. Record the new sum. Document why the sum changed even though Session A never modified the data.
4. **Cleanup:** Type ROLLBACK; in Session A.

Part B: Fixing the Anomaly

1. **Session A:** Change isolation level to SERIALIZABLE. Start a transaction and run the SUM query again.
2. **Session B:** Start a transaction. Try to insert a different new account (204, 'Amit', 5000.00).
3. **Document:** What exactly happens in Session B's terminal window? Why is it behaving this way?
4. **Session A:** Type COMMIT;.
5. See What occurs in Session B as a result of Session A committing

SQL Syntax Hints :

SQL

-- Setting Isolation Levels Syntax:

SET SESSION TRANSACTION ISOLATION LEVEL SERIALIZABLE;

-- Options: READ COMMITTED, REPEATABLE READ, SERIALIZABLE